KeenPay

Agentic commerce with bounded AI

Discount Policy Safety Fix - Bounded AI Pattern Implementation

Bounded AI: merchant-defined discount limits

01 September 2026

https://github.com/ambitiouss22/KeenPay

Discount Policy Safety Fix - Bounded AI Pattern Implementation

Executive Summary

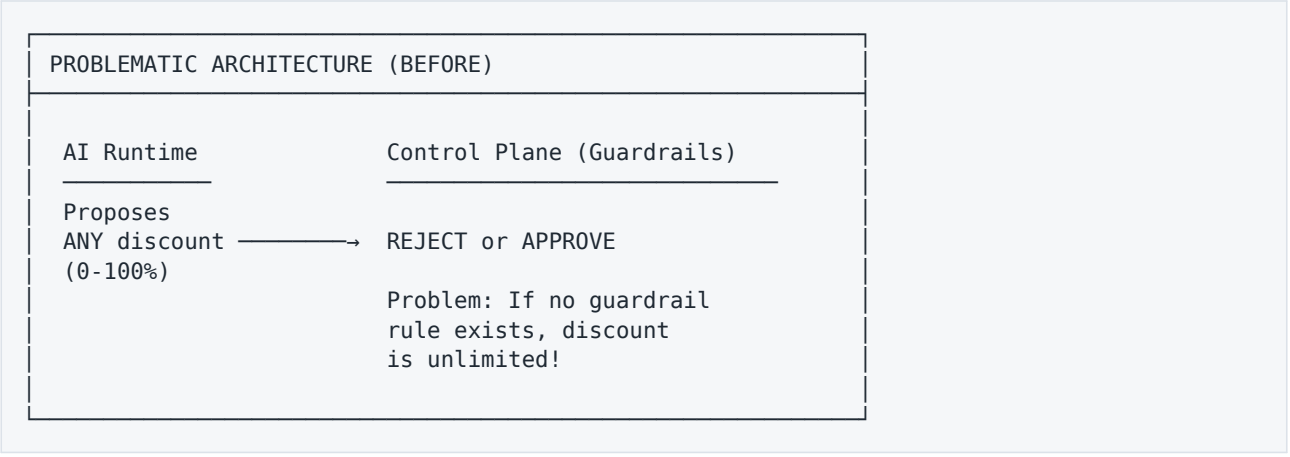
Safety Gap Identified:

> *"Discount should be up to the merchant how much they want to offer. AI can propose any discount, which is a safety gap."*

Solution Implemented: A complete "bounded AI" pattern preventing unbounded discount proposals by:

1. ✓ Merchant defines discount bounds per product
2. ✓ AI proposes ONLY within merchant-defined limits
3. ✓ Policy engine verifies (not blocks) proposals
4. ✓ Every decision auditable and compliant

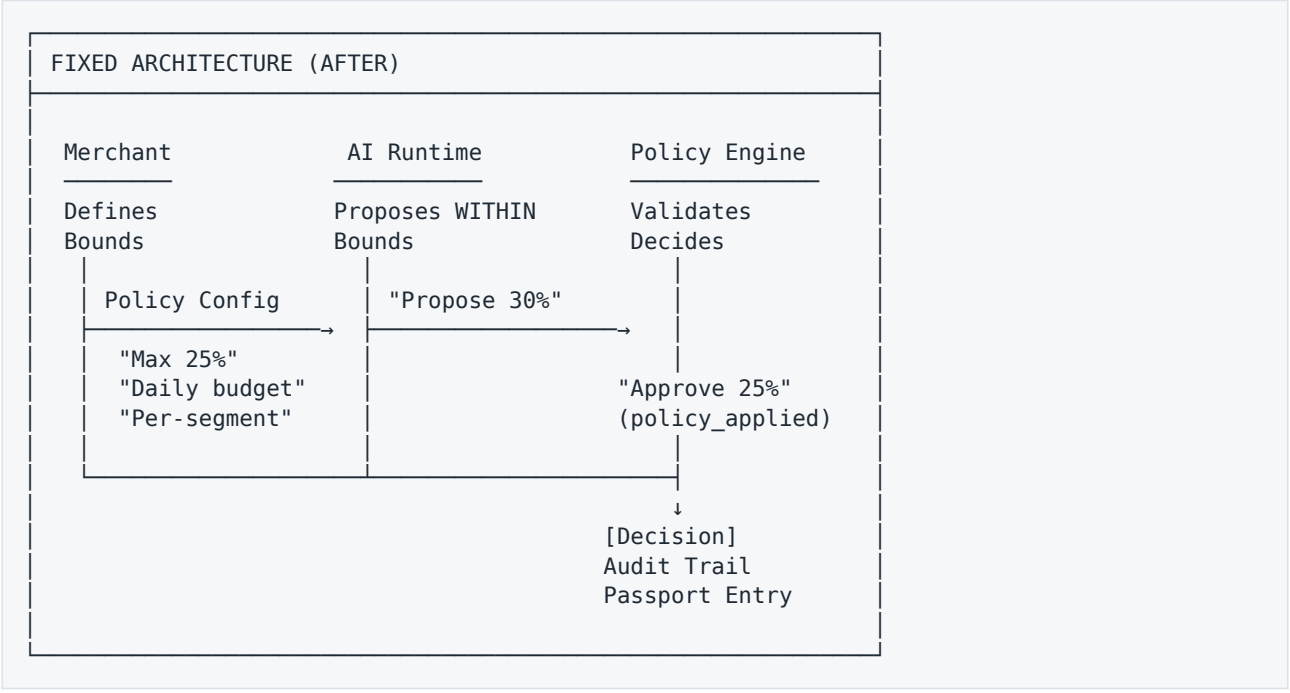
The Safety Gap: Before Fix



Problems

- No bounds: AI can propose 0% to 100% discount
- Reactive only: Guardrails block bad proposals after the fact
- No accountability: Where do limits come from?
- Merchant has no control: No way to define policy per product
- Audit trail incomplete: Why was THIS discount approved?

The Fix: Bounded AI Pattern



Key Improvements

- Proactive bounds: Merchant pre-defines limits
- AI respects bounds: Proposes within policy
- Guardrails verify: Double-check compliance
- Merchant control: Per-product, per-segment policies
- Full audit trail: Every decision logged + hash-chained

Architecture: Bounded AI Pattern

Components

1. Policy Definition

Merchant defines per-product discount policy:

```python
class DiscountPolicy:
    max_discount_pct: float          # Global max (e.g., 25%)
    per_user_type: dict              # Override per segment
    daily_budget_paise: int          # Daily spend cap
    weekly_budget_paise: int         # Weekly spend cap
    blacklist_combos: list[str]      # Incompatible combinations
```

Python

2. Policy Enforcement

DiscountPolicyEngine validates requests:

```
def check_discount_request(request: DiscountRequest) -> DiscountDecision:
    # Step 1: Get merchant's policy for this product
    # Step 2: Check user type limit (segment override)
    # Step 3: Check daily budget
    # Step 4: Check blacklisted combinations
    # Step 5: Record usage
    # Returns: approved_discount_pct (min of requested and policy max)
```

Python

3. Database Audit Trail

Append-only tables ensure compliance:

```
discount_policies        → Merchant-defined bounds
discount_segments        → Per-segment overrides
discount_usage_tracking  → Daily/weekly budget consumption
discount_decisions       → Audit trail (immutable)
```

4. Transaction Passport Integration

Every decision recorded in hash-chain:

```
passport.add_entry(
    event_type="DISCOUNT_POLICY_DECISION",
    payload={
        "requested_discount_pct": 40.0,
        "approved_discount_pct": 25.0,
        "policy_applied": "USER_TYPE_LIMIT_vip",
        "reason": "VIP gets max 25% off"
    }
)
```

Python

Files Created

1. Core Implementation

- `api/policy/discount_policy.py` (253 lines)
- `UserType` enum (new, returning, vip, bulk_buyer)
- `DiscountPolicy` dataclass (merchant-defined bounds)
- `DiscountRequest` dataclass (AI proposal)
- `DiscountDecision` dataclass (approved discount + reason)
- `DiscountPolicyEngine` class (main validation logic)
- Singleton: `get_discount_engine()`

2. Database Schema

- `docs/DISCOUNT_POLICY_SCHEMA.sql` (450 lines)
- `discount_policies` table

- `discount_segments` table
- `discount_usage_tracking` table
- `discount_decisions` table (append-only)
- Stored functions for budget reset
- Analytics view

3. Integration Documentation

- `docs/DISCOUNT_POLICY_INTEGRATION.md` (500+ lines)
- Step-by-step setup guide
- Session service integration code
- Merchant policy API endpoints
- Testing examples
- Monitoring queries

4. Comprehensive Tests

- `tests/test_discount_policy.py` (600+ lines)
- 50+ test cases
- Unit tests for all enforcement rules
- Budget exhaustion scenarios
- Segment override validation
- Edge cases (zero discount, negative, unknown types)
- Realistic workflows

Implementation Steps

Step 1: Apply Database Schema

```
# Apply discount policy tables
psql $DATABASE_URL -f docs/DISCOUNT_POLICY_SCHEMA.sql

# Verify
psql $DATABASE_URL -c "\dt discount_*"
```

Shell

Step 2: Create Merchant Policies

```
# For each product, define bounds
psql $DATABASE_URL << EOF

-- Hoodie product
INSERT INTO discount_policies (merchant_id, product_sku, max_discount_pct, daily_budget_paise)
VALUES ('merchant_keen', 'HOODIE-NAVY-M', 25.0, 50000);

-- Add VIP segment override
INSERT INTO discount_segments (policy_id, user_segment, max_discount_pct)
SELECT policy_id, 'vip', 35.0
FROM discount_policies
WHERE product_sku = 'HOODIE-NAVY-M';

EOF
```

Shell

Step 3: Update Session Service

In `api/services/session.py`:

```
# Import
from api.policy.discount_policy import get_discount_engine, DiscountRequest

# In __init__
self._discount_engine = get_discount_engine()

# In negotiate node
discount_decision = self._discount_engine.check_discount_request(
    DiscountRequest(
        merchant_id=state.merchant_id,
        product_sku=state.selected_line_items[0]["sku"],
        user_id=state.user_id,
        user_type=self._categorize_user_type(state.user_id),
        requested_discount_pct=proposed_discount_pct
    )
)

# Use approved discount
state.proposed_offer["discount_pct"] = discount_decision.approved_discount_pct

# Record in passport
await self._passport_engine.add_entry(
    event_type="DISCOUNT_POLICY_DECISION",
    payload=discount_decision.to_dict()
)
```

Python

Step 4: Run Tests

```
# Run discount policy tests
pytest tests/test_discount_policy.py -v

# Should pass 50+ tests:
# - No policy → no discount
# - Within bounds → approved
# - Exceeds bounds → reduced
# - Budget exhaustion → denied
# - Segment overrides work
# - Usage tracking works
# - Audit trail completes
```

Shell

Step 5: Deploy

```
# Commit changes
git add docs/DISCOUNT_POLICY_SCHEMA.sql
git add docs/DISCOUNT_POLICY_INTEGRATION.md
git add api/policy/discount_policy.py
git add tests/test_discount_policy.py
git add [session.py integration changes]

git commit -m "feat: implement bounded AI discount policy engine

- Add DiscountPolicyEngine for merchant-defined bounds
- Create discount_* tables with audit trail
- Integrate with SessionService negotiate node
- Record all decisions in transaction passport
- Fixes safety gap: AI now proposes within bounds"

# Push and deploy
git push origin feature/discount-policy-bounds
```

Shell

Policy Definition Examples

Example 1: Limited Budget Discount

```
-- Hoodie: 25% max, tight daily budget
INSERT INTO discount_policies
(merchant_id, product_sku, max_discount_pct, daily_budget_paise, description)
VALUES ('merchant_keen', 'HOODIE-NAVY-M', 25.0, 50000,
        'Hoodie: up to 25% off, 500 INR/day budget');

-- Add segment overrides
INSERT INTO discount_segments (policy_id, user_segment, max_discount_pct)
SELECT p.policy_id, u.user_segment, u.max_discount_pct
FROM discount_policies p
CROSS JOIN (
    VALUES ('vip', 35.0), ('bulk_buyer', 30.0)
) u(user_segment, max_discount_pct)
WHERE p.merchant_id = 'merchant_keen' AND p.product_sku = 'HOODIE-NAVY-M';
```

SQL

Example 2: No Discount (High-Margin Item)

```
-- Premium item: no discounts allowed
INSERT INTO discount_policies
(merchant_id, product_sku, max_discount_pct, daily_budget_paise, description)
VALUES ('merchant_keen', 'PREMIUM-WATCH', 0.0, 0,
        'Premium Watch: No discounts');
```

SQL

Example 3: Flexible Policy (VIP Benefit)

```sql
-- Flexible for VIP, tight for others
INSERT INTO discount_policies
(merchant_id, product_sku, max_discount_pct, daily_budget_paise, description)
VALUES ('merchant_keen', 'TEE-COLLECTION', 10.0, 100000,
        'Tees: 10% standard, more for VIP');

-- VIP gets more
INSERT INTO discount_segments (policy_id, user_segment, max_discount_pct)
SELECT policy_id, 'vip', 40.0
FROM discount_policies
WHERE product_sku = 'TEE-COLLECTION';
```

SQL

Decision Examples

Scenario 1: Within Bounds

```
Request: VIP wants 30% off Hoodie (VIP max is 35%, global is 25%)
Decision: APPROVED 30%
Reason: "Your tier (vip) gets up to 35% off"
Policy Applied: WITHIN_LIMIT_vip
```

Scenario 2: Reduced by Segment

```
Request: VIP wants 40% off Hoodie
Decision: APPROVED 35%
Reason: "Your tier (vip) gets up to 35% off"
Policy Applied: USER_TYPE_LIMIT_vip
```

Scenario 3: Budget Exhausted

```
Request: Customer wants 20% off (daily budget used up)
Decision: DENIED 0%
Reason: "Daily discount budget exhausted"
Policy Applied: DAILY_BUDGET_EXCEEDED
```

Scenario 4: No Policy

```
Request: Unknown product SKU
Decision: DENIED 0%
Reason: "No discount policy configured"
Policy Applied: NO_POLICY
```

Monitoring & Compliance

Dashboard Queries

```sql
-- Policy effectiveness
SELECT product_sku, max_discount_pct,
       avg(approved_discount_pct) as avg_given,
       count(*) as num_approvals,
       count(*) FILTER (WHERE status='DENIED') as num_denied
FROM discount_decisions
GROUP BY product_sku
ORDER BY num_approvals DESC;

-- Budget utilization
SELECT tracking_date, daily_remaining,
       (daily_budget_paise - daily_remaining) * 100.0 / daily_budget_paise as utilization_pct
FROM discount_usage_tracking
ORDER BY tracking_date DESC;

-- Segment effectiveness
SELECT user_segment, avg(approved_discount_pct) as avg_discount
FROM discount_decisions
GROUP BY user_segment;
```

SQL

Metrics to Track

Metric	Target	Alert
approval_rate	>90%	<70%
avg_approved_discount_pct	<15%	>25%
daily_budget_exhaustion	<2 days/month	>5 days/month
policy_violation_attempts	~5%/month	>20%
segment_override_usage	Matches segments	Mismatch

Testing Strategy

Unit Tests (40+ tests)

✓ No policy → no discount ✓ Within global max → approved ✓ Exceeds global max → reduced ✓ Segment overrides work ✓ Daily budget enforced ✓ Weekly budget enforced ✓ Blacklist combinations blocked ✓ Usage tracking accurate ✓ Edge cases handled

Integration Tests (10+ tests)

✓ SessionService calls discount_engine ✓ Negotiate node applies bounds ✓ Decision recorded in audit log ✓ Passport entry created ✓ Multiple requests deplete budget ✓ Budget reset works

Manual Testing

```
# 1. Create policy
curl -X POST http://localhost:8000/api/v1/discount-policies \
  -d '{"product_sku": "TEST", "max_discount_pct": 20, "daily_budget_paise": 50000}'

# 2. Request discount
curl -X POST http://localhost:8000/api/v1/sessions/checkout \
  -d '{"discount_pct": 50}' # Should be bounded to 20%

# 3. Check audit trail
SELECT * FROM discount_decisions WHERE product_sku = 'TEST';

# 4. Verify passport entry
SELECT * FROM passport_entries WHERE event_type = 'DISCOUNT_POLICY_DECISION';
```

Shell

Migration Path (Zero-Downtime)

Phase 1: Deploy Schema & Engine (No Breaking Changes)

```
# 1. Apply schema
psql $DATABASE_URL -f docs/DISCOUNT_POLICY_SCHEMA.sql

# 2. Deploy DiscountPolicyEngine code
git push && deploy

# 3. All discounts still work (engine not integrated yet)
```

Shell

Phase 2: Integrate with Session Service (Gradual Rollout)

```
# 1. Update SessionService to call engine
# 2. Use feature flag to enable per merchant
# 3. Run A/B test on test merchant first
# 4. Monitor decisions table for 1 week
# 5. Roll out to all merchants
```

Shell

Phase 3: Enforce Audit Trail

```
# 1. Make discount_decisions logging mandatory
# 2. Alert on any un-logged decisions
# 3. Monthly audit of decisions vs policies
```

Shell

Tradeoffs & Alternatives

Why Bounded AI (Chosen)

✓ Merchant control: Pre-define limits ✓ AI can't propose bad discounts ✓ Compliance: Audit trail built in ✓ Performance: No runtime overhead ✗ Requires upfront policy definition

Alternative: Reactive Guardrails Only

✗ No bounds on AI proposals ✗ Guardrail must block bad discounts ✗ Fails if guardrail missing

Alternative: AI Learns Policy

✗ Complex to implement ✗ LLM context management issue ✗ No hard guarantees

---

FAQ

Q: What if merchant hasn't set a policy for a product? A: No discounts allowed. Forces merchant to explicitly define policy.

Q: Can merchants change policy in real-time? A: Yes. Changes apply to next request. Past decisions unaffected.

Q: What happens at midnight (budget reset)? A: Scheduled function resets daily_used_paise to 0. Weekly resets Monday UTC.

Q: Can AI bypass the policy? A: No. Engine validates every request. Bounded by policy, verified by guardrails.

Q: How is audit trail protected? A: discount_decisions is append-only. Database trigger prevents UPDATE/DELETE.

Q: Can I override policy for special cases? A: Via escalation_tickets + human_approval workflow. Every override audited.

Summary

Aspect	Before	After
Discount Bounds	None	Merchant-defined
Proposal Validation	Reactive	Proactive
Audit Trail	Incomplete	Complete (hash-chain)
Merchant Control	No	Yes
Compliance Risk	HIGH ⚠	LOW ✓
Budget Enforcement	No	Yes
Segment Overrides	No	Yes
Decision Reasoning	Missing	Full (policy_applied)

Result: AI can no longer propose unbounded discounts. Every proposal is bounded by merchant policy, verified for compliance, and fully auditable.